IR-Agent: A Tool-Using AI Agent for Information Retrieval

5. Submission Links

- **GitHub repository:** https://github.com/sherazahmad634/Information-Retrieval-AI-agent
- **Demo video:** https://drive.google.com/file/d/1tldekOih3oVzLI3dtG2wFdQWjlt0NvHk/view?usp=sharing

Abstract

This report describes **IR-Agent**, a production-quality Retrieval-Augmented Generation (RAG) question-answering system built from scratch as the deliverable for an Information Retrieval coursework module.

The system extends a conversational Large Language Model (LLM) with five capabilities that materially improve grounded answer generation:

1. Hybrid lexical and dense retrieval
2. Cross-encoder re-ranking
3. Tool-using function-calling agent loops
4. Two-tier conversational memory
5. Provider-agnostic LLM backend support

The agent answers user questions using only the indexed corpus and provides citations for retrieved evidence. The system architecture combines BM25 lexical retrieval, dense vector embeddings, Reciprocal Rank Fusion (RRF), and cross-encoder reranking to improve retrieval precision.

This report summarises the overall design, highlights several novel implementation choices, and reflects on the use of AI coding assistants during development.

1. Introduction

Large Language Models (LLMs) are powerful conversational systems; however, they may hallucinate facts and cannot inherently access private or external documents. Retrieval-Augmented Generation (RAG) addresses these limitations by grounding model responses in retrieved external knowledge at inference time.

The coursework brief required building an AI agent extended with Information Retrieval capabilities such as document search, memory, web search, and tool usage. This project addresses those requirements by implementing an end-to-end system that:

- ingests heterogeneous document formats,
- indexes them using hybrid retrieval methods,
- exposes retrieval through a tool-using agent architecture,
- and produces grounded, cited responses.

2. System Architecture

The backend is implemented as a FastAPI service exposing a tool-using agent through REST and Server-Sent Events (SSE) APIs.

The system supports ingestion of:

- PDF
- DOCX
- HTML
- Markdown
- Plain text documents

Documents are parsed, normalised, and split into sentence-aware overlapping chunks of approximately 512 tokens with a 64-token overlap.

Dense retrieval is implemented using:

- Sentence Transformers (`all-MiniLM-L6-v2`)
- ChromaDB vector storage

A second in-memory BM25-Okapi index provides lexical retrieval over the same chunks.

The two ranked retrieval outputs are combined using Reciprocal Rank Fusion (RRF), after which top candidates are reranked using the cross-encoder model:

- `ms-marco-MiniLM-L-6-v2`

This hybrid retrieval pipeline significantly improves retrieval quality compared to standalone dense retrieval systems, particularly for acronym-heavy and rare-term queries.

---

3. Tool-Using Agent Design

The agent operates through an OpenAI-format function-calling loop with a six-iteration safety cap.

The following tools are registered:

- `document_search`
- `document_ingest`
- `web_search`
- `calculator`
- `remember`
- `recall`

Additional Components

Working Memory

A bounded per-session conversational memory buffer maintains short-term conversational context.

Long-Term Memory

Durable memory is stored in SQLite as a key-value store and accessed only through explicit `remember` and `recall` tools.

Provider-Agnostic Backend

The system supports any OpenAI-compatible API endpoint through the `OPENAI_BASE_URL` override, enabling compatibility with:

- OpenAI
- Groq
- Microsoft
- Ollama

This design avoids vendor lock-in and improves reproducibility.

---

4. Novel Design Choices

Three implementation choices distinguish the system from a baseline RAG pipeline.

4.1 Hybrid Retrieval with Cross-Encoder Re-ranking

Many student RAG systems rely exclusively on dense vector similarity. However, dense retrieval often struggles with:

- acronyms,
- exact terminology,
- and rare keywords.

By combining BM25 lexical retrieval with dense embeddings and second-stage cross-encoder reranking, the system achieves substantially higher retrieval precision.

4.2 Two-Tier Memory Architecture

The system separates memory into:

- short-term conversational working memory,
- and explicit long-term memory tools.

This prevents implicit, uncontrolled memory accumulation and improves transparency and auditability.

4.3 Provider-Agnostic Deployment

The same codebase supports multiple inference providers without modification. During development, the system was validated across:

- OpenAI APIs
- Groq APIs

- Ollama local inference

This improves flexibility and deployment portability.

---

5. Reflection on AI Coding Assistants

Anthropic Claude was used extensively as a pair-programming assistant during development.

Its major strengths included:

- rapid FastAPI scaffolding,
- Pydantic schema generation,
- retrieval pipeline guidance,
- and recalling standard Information Retrieval concepts such as:
    - BM25 behaviour,
    - Reciprocal Rank Fusion,
    - and reranking pipelines.

However, several limitations emerged:

- hallucinated method signatures,
- incorrect FastAPI assumptions,
- invalid SSE parsing logic,
- and provider-specific incompatibilities.

For example:

- Groq rejected `null function_call` values accepted by OpenAI.
- SSE parsing initially failed due to CRLF line-ending assumptions.

All issues were eventually identified through testing and end-to-end validation.

The most valuable contribution of AI coding assistants was the development speed. However, effective review and correction still require strong prior knowledge in Information Retrieval and backend engineering. AI tooling amplified productivity but did not replace technical expertise.

References

1. Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). *Reciprocal rank fusion outperforms Condorcet and individual rank learning methods*. SIGIR 2009.
2. Karpukhin, V., Oğuz, B., Min, S., et al. (2020). *Dense passage retrieval for open-domain question answering*. EMNLP 2020.
3. Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for knowledge-intensive NLP tasks*. NeurIPS 2020.
4. Robertson, S. E., & Zaragoza, H. (2009). *The probabilistic relevance framework: BM25 and beyond*.
5. Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., & Gurevych, I. (2021). *BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models*.
6. Yao, S., Zhao, J., Yu, D., et al. (2022). *ReAct: Synergizing reasoning and acting in language models*. arXiv:2210.03629.